Claude Code mods: panes, watchers, task ledgers and session handoffs

Choose the capability before installing

Claude Code mods change behavior or the interface through event handlers. Across six sources, the useful tasks are navigation and appearance, visibility across sessions, unfinished-work tracking, overlooked-point detection and session transfer. They belong in one guide, while each original transcript, URL and dated download remains available.

Need	Capability	Evidence required
Navigate files or change presentation	A per-session pane or skin	Correct file paths and supported rendering hooks
See several sessions	A local dashboard and explicit transport	Session identity, timestamps, expiry and reconnect behavior
Keep unfinished tasks visible	A task ledger	Declared requirement, actual execution evidence and unresolved state
Notice an overlooked point	You Should Know	An eligible session and a useful side-agent note
Continue after context fills	A structured handoff	A saved brief and correct resumption against an answer key

The official mod overview (https://code.claude.com/docs/en/plugins/mods/overview) describes JavaScript/TypeScript handlers inside plugins. A pane does not by itself create shared storage, and reassuring text does not prove a task passed its checks.

Start with a compatible, inspected plugin

Check the installed build, read the selected plugin's handlers, validate it, then load it in an isolated session. Examples are learning material rather than a production guarantee. Use the current plugin installation documentation (https://code.claude.com/docs/en/discover-plugins) and official sample overview (https://code.claude.com/docs/en/plugins/mods/overview#try-a-sample-mod) to choose a concrete example.

```
claude --version
claude plugin validate ./chosen-plugin
claude --plugin-dir ./chosen-plugin
```

Replace chosen-plugin with an inspected local directory. No plugin or example was installed as part of this research.

Watch for overlooked work without confusing it with memory

You Should Know (https://code.claude.com/docs/en/interactive-mode#you-should-know-suggestions) is an off-by-default built-in mod on compatible releases. Its side request has no tools. It can flag a missed tradeoff; it cannot independently inspect a repository, run tests or guarantee that task state is preserved. Feature-flag eligibility can explain a missing plugin, including sessions where telemetry is disabled. Keep that user preference separate from installing a tool.

```
/plugin enable cc-plugin-you-should-know@builtin
```

The older guides' uncertainty about this built-in feature is retained as dated evidence. The current official documentation takes precedence.

Transfer a long session with an explicit brief

The new source adds claude-auto-handoff (https://github.com/alexknowshtml/claude-auto-handoff), a community mod that replaces automatic compaction with a brief, clear and reseed. Its editable template and facts-only fallback are useful, but summaries remain lossy. Default thresholds are project choices, not universal settings for every context window.

```
git clone https://github.com/alexknowshtml/claude-auto-handoff.git claude-auto-handoff
claude plugin validate ./claude-auto-handoff
claude --plugin-dir ./claude-auto-handoff
```

Review the cloned code and record a commit before relying on it. Use a disposable session first. Model requests consume allowance, and the optional brief viewer can expose the brief to a configured tailnet. Current upstream notes include Windows fallbacks; check the viewer's shell requirements separately.

Create a ground-truth note with decisions, abandoned approaches, the latest unanswered request and the next step. After an actual handoff, check both the brief and resumed behavior. Record omissions, invented details, repeated work, timing and additional model usage. This guide has not executed that experiment.

Build a ledger before a dashboard

Filetree and skins concern navigation and appearance: filetree (https://github.com/data-goblin/claude-code-filetree), skins (https://github.com/hellosverre/claude-skins) and Reflect (https://github.com/BayramAnnakov/claude-reflect) remain separate community choices.

Duckboard-style aggregation needs a documented local event transport. Left Undone-style tracking needs requirements and execution evidence, not only TODO detection. Build one session adapter first, then test malformed events, stale sessions and reconnects before adding another session. Caching economics remain in the separate caching topic; a UI counter does not make requests free.

Reusable project brief

```
Design a read-only local session ledger with a structured handoff evaluation.
Start with fixture events and one inspected mod adapter. Track declared tasks,
evidence links, unresolved requests and session identity separately.
Keep transport and viewer exposure disabled until explicitly configured.
Use fictional seeded decisions and compare manual notes with an actual handoff.
Report missing facts and unavailable measurements honestly. Do not infer test
success from prose or auto-approve any commands. Deliver a small pane, fixtures,
setup instructions and an evidence-backed report before a multi-session UI.
```

Next step and scope

Choose one capability from the table, verify compatibility and run its smallest isolated demonstration. The main guide organizes all six sources as one workflow; individual PDFs retain their original dates and may contain older implementation details. Proposed projects and documented commands have not been executed here.

Original sources

- https://youtu.be/ERNqRK6sMJY
- https://www.instagram.com/reel/DeRQMvaBZY4/
- https://youtu.be/lDrAZ1wAyVs
- https://www.instagram.com/reel/DeJhQj8hkGy/
- https://www.instagram.com/reel/DeCu7c8BZmd/
- https://www.instagram.com/reel/Deo8i2rEQi5/